EgoQuant: Project Summary

2026 Shenzhen FinTechathon, WeBank International Track, Topic B: Wealth Advisory Agent. Status: 20 October 2026. Live system: https://fintechaton-frontend.vercel.app (API: https://web-production-ee5cc.up.railway.app, interactive docs at `/docs`).

1. In one paragraph

EgoQuant is a wealth-advisory agent that gives a person a portfolio they can **understand, question and audit**. A formal risk questionnaire and a short Risk Game find out how much risk the person can afford and how they actually behave about it; a team of specialised agents turns that, the person's interests and real market data into a portfolio and a plain-language report; an independent Risk Engine re-checks every limit and has the final word; and an AI advisor explains everything, runs "what if" scenarios and checks possible trades, **without ever being allowed to decide anything**. Confirmed advice can be sealed on a public blockchain so that any later change is detectable. The design rule that organises the whole system: **numbers that matter come from deterministic, tested code; AI only explains.**

2. The three promises (and how to see each in two minutes)

Promise	What it means	Where to see it
Fit	The portfolio is suitable for this person: their category is the *lower* of what their finances allow and how they feel about risk; their real behaviour can only make it safer.	Questionnaire, then Risk Game, then the "How your category was set" sentence on the report
Proof	Every number has a reason and a source; every limit is shown; an independent engine re-checks them; AI answers are checked against the engine's facts.	Report tabs Limits and Why; the chat; "Approved by the Risk Engine"
Trace	What we advised can be shown unchanged later.	Confirm and seal, then **Verify a report** (works without an account)

3. How this answers the judging criteria

Criterion (weight)	What we built	Details
Task completion (40%)	The whole advisory chain for a real person: questionnaire, universe of allowed products with real prices, candidate portfolios inside hard limits, behavioural tightening, a Monte Carlo goal analysis ("N in 100 simulated futures"), an independent risk check, an HTML/JSON report, a news briefing per holding, accounts with saved reports and imported broker history, a trade check, an AI advisor with what-if and trade questions.	Section 5-7; `docs/judges/05_USER_GUIDE_AND_DEMO.md`
Security and compliance (30%)	Suitability by design (attitude versus capacity, product risk levels, knowledge gate), per-user isolation with Row-Level Security, minimal and expiring data with complete erasure, a fenced AI (whitelisted tools, number grounding, redaction, compliance filter), tamper-evident records, graceful failure, about 3,500 automatic tests including hostile-input suites.	`docs/judges/02_SECURITY_COMPLIANCE_REPORT.md`; `docs/SECURITY.md`

Criterion (weight)	What we built	Details
Innovation and interaction (30%)	The Risk Game as one third of the attitude to risk; behaviour-aware tightening from a real IBKR trade history; a regime-aware goal simulation; an AI advisor that runs the real engine for what-if and trade questions yet cannot invent a number; on-chain sealing with a public verifier; a Method tab that shows how the portfolio was chosen; an optional second construction method (risk parity, hierarchical risk parity, Black-Litterman, resampling) with an honest acceptance table.	Section 8

4. The person's journey (what a judge sees)

1. **Home and sign-up.** A short explanation and an account (e-mail and password; the judges' demo account is described in the user guide). Visitors can use the Advisor and the Verify page without an account.
2. **I. Profile: the questionnaire (25 questions; the counter shows 26 steps because one question asks for two amounts).** 13 risk-attitude items (Grable and Lytton, verbatim); a capacity section (the goal for the money, when it will be needed, the amount invested and how much must come back, months of expenses saved outside it, its share of total wealth, income stability); a knowledge and experience section (three Lusardi and Mitchell questions verbatim, the products invested in before, years of investing); and one question on whether a fall in value could be accepted (a "no" gives cash-like products only). The result shows the category C1 (most careful) to C5 and why.
3. **II. Risk Game (about two minutes, offered to everyone).** Seven rounds of hypothetical choices (play money) and a crash test. It counts one third of the attitude to risk; the final category can never pass the capacity cap. Skipping is allowed after a confirmation.
4. **III. Investments.** The person picks interest areas and trading platforms. Only products their category allows are offered; a counter shows how many of the (up to eight) areas will be used for prices and news.
5. **IV. Portfolio.** A goal (amount, years, monthly saving) and optionally an IBKR trade history (read-only upload or Flex token). The build takes a few seconds on the saved real-price snapshot.
6. **The report** (tabs, in the order of the site): **Summary** (allocation, approved by the Risk Engine), **Goal** (the chance of reaching it, what would change it), **Outlook** (fan chart of simulated futures), **Holdings**, **Limits** (every limit with its source and the Risk Engine's result), **Stress** (hypothetical shocks), **Trade check** (one planned trade against the limits), **News** (per-holding briefing), **Data** (where each number comes from), **Method** (how the choice was made, including the Risk Game sentence) and **Why** (the reasons, with a pointer to ask the Advisor).
7. **Advisor chat.** Plain-language answers about the report; "what if I add 200 a month?" runs the real engine; "could I sell 20% of VTI?" runs the deterministic trade check on the saved trading history (signed-in investors).
8. **Trade check tab.** The same check with a form, in the portfolio; a signed-in person can record a trade they made (EgoQuant never trades).
9. **Account.** Saved reports, imported history, consents, activity log, delete my data.
10. **Confirm and seal, then Verify.** Confirm an approved plan: the exact report, marked "Sealed for" your username, is fingerprinted and only the fingerprint goes on the Sepolia test chain. Download the sealed report and check it (or a tampered copy) on **Verify a report**: it says "Unchanged" or "Does not match".

5. Architecture

```
Questionnaire + Risk Game --> category C1..C5 + numeric limits             (deterministic code)
        |
        +--> Agent C  market intelligence: allowed universe, real prices, data provenance
        |
        +--> Agent A  quant: ~100 candidate portfolios inside the limits, goal simulation
                 ^   |
   (tighten only)|   v
             Agent B  behavioural twin: trade history, crash test, game --> numeric limits
                     |
                     v
             Risk Engine (independent re-check, final say) --> Client report (JSON + HTML)
                     |
   Agent D  read-only news briefing per holding (one labelled AI exception: the tone label)
   Advisor  explains sealed facts; tools: what-if, check one trade; number grounding; compliance filter
```

Component	Role	Code
Risk questionnaire and Risk Game blend	Category and limits	`egoquant/risk_profile/`
Agent C (market intelligence)	Instrument catalogue with R1-R5 product levels, allowed universe per category and platform, price sources (a committed snapshot of 292 tickers plus live feeds with a time budget), data provenance	`egoquant/agents/market_intel/`
Agent A (quant)	Constrained optimiser, candidate menu (about 100 portfolios across stock levels and styles), Monte Carlo goal analysis	`egoquant/agents/quant/`, `egoquant/regime/`
Agent B (behavioural twin)	Reads the IBKR history (selling in falls, turnover), the crash test and the game; returns numeric limits that can only tighten; screens the candidates	`egoquant/agents/behavioral/`
Agent D (market briefing)	News per holding, fund-aware tone, source tiers, "what changed" (never changes a number)	`egoquant/briefing/`
Advisor (Agent E)	Chat, what-if, trade questions, trade-check explanations	`chat.py`, `egoquant/advisor/`, `egoquant/agents/what_if.py`, `egoquant/agents/trade_chat.py`
Risk Engine	Independent final check	`egoquant/risk_engine.py`
Report	Client report and HTML	`egoquant/report.py`
Accounts and seals	Supabase Auth and Postgres with RLS; Sepolia fingerprints	`egoquant/accounts/`, `supabase/`, `egoquant/anchoring/`
Frontend	Next.js 16, React 19, Tailwind, shadcn; server route handlers hold the session cookies	`fintechaton-frontend/`

Deployment: the frontend on Vercel, the backend on Railway (Python 3.14), accounts on Supabase. A read-only `python eval/deploy_check.py <backend url>` verifies a deployment.

6. How a portfolio is built (plain words)

1. **Category and limits.** From the questionnaire and game: the category (C1-C5), the highest product risk level allowed, the maximum share in stocks, gold and commodities, crypto and cash, a volatility cap and a loss limit in a named hypothetical stress scenario.

2. **Universe.** Agent C selects the products the person may hold (right risk level, right platform, passing the knowledge check for complex products), fetches or loads prices, and records where every price came from.
3. **Candidates.** Agent A builds about 100 distinct portfolios: 20 points on the risk axis (the share in stocks from zero up to the category's cap) times 5 construction styles (lowest risk, diversified, equal weight, risk-balanced, tilted toward the person's interests), each solved under every hard limit (`candidate_levels`, `candidate_target` in `egoquant/models.py`).
4. **Behavioural screen.** Agent B turns the person's real trade history (for example repeated selling in falls), the crash test and, for the category only, the game into a comfort band for the loss in a severe fall, and keeps between 5 and 20 candidates inside it (`screen_min_keep`, `screen_max_keep`). It can only tighten.
5. **Goal simulation.** Each kept candidate is simulated against the person's goal on 10,000 paths (the same paths for every candidate, a fixed seed). A path chains month-long blocks drawn from about 20 years of real daily returns. By default a regime-aware sampler decides which kind of month comes next (a four-state Hidden Markov model fitted offline on market history, hash-checked, run as plain numpy, with a fall-back to the plain block bootstrap and a kill switch), so calm periods and crises last as long as they did in the past. Two corrections keep history's luck out of the answer: bond and cash funds get the interest rate the market pays today (plus a term premium for bonds) instead of their past average, and stocks, equity funds and crypto keep every real daily move but get the average that their sensitivity to the world stock market earned, because today's most-traded names are today's winners. The chosen portfolio is the one with the highest chance of reaching the goal; among portfolios within one percentage point of the best, the one with the smaller shortfall in the worst 5% of paths, then the lower volatility (`egoquant/agents/quant/goal.py`, `rank`).
6. **Independent check.** The Risk Engine recomputes every limit from the weights alone; only an approved portfolio is shown.
7. **Report.** Every number comes with its reason, limit source and a hypothetical-not-forecast label; the chance is expressed as "N in 100 simulated futures".

7. The Risk Game rule in one box

Attitude to risk (1-5) = two thirds of the questionnaire's level + one third of the Risk Game's level, rounded. Capacity (1-5) = what the person's finances allow (money needed back and the horizon are hard caps). ***Category = the lower of the two.*** *"I could not accept any loss" always gives C1. A real trade history, the crash test and the game can only tighten the portfolio further.* (`docs/RISK_GAME_BLEND.md`)

8. Innovation highlights

1. **Behaviour counts, safely.** The Risk Game gives every person a behavioural signal (not only those with a trade log); the trade log, when present, is stronger and tightens only.
2. **An AI that can run the engine but cannot decide.** Two whitelisted tools (what-if, check one trade) execute deterministic code; every number in the answer is checked against the engine's facts; personal data never reaches the model.
3. **Tamper-evident advice.** Salted on-chain fingerprints with a public verifier and demonstrations of detected tampering.
4. **A goal simulation that is honest about history.** Regime-aware sampling, rate anchoring, a hindsight correction for equities, a calibration backtest on rolling start dates (it improved the score against the plain bootstrap by 0.007 at five years and 0.011 at ten years over all dates, but on the later

confirmation dates the gain is within the noise: `eval/results/regime_v2_summary.md`), and results as "N in 100".

5. **Explainable by construction.** The Method tab, the limits table with sources, the category sentence, fixed-template reasons.
6. **An optional second construction method** (risk parity, hierarchical risk parity, maximum diversification, Black-Litterman, resampled efficiency) built, tested and held back by our own acceptance rule; the decision sheet shows the numbers (`docs/V2_DECISION_SHEET.md`).
7. **Resilience engineering.** Price-feed time budgets, a snapshot mode, kill switches, a deploy check, and a drill that simulates a feed where every request takes 15 seconds.

9. What is real, what is simulated, what is invented

Item	Status
Prices	**Real** historical prices (a committed snapshot to 16 Oct 2026: 292 tickers, about 19 years; live feeds when allowed). Every report names the source and the date.
Goal chances	**Simulations on past data**, labelled hypothetical, never forecasts
Stress losses	**Hypothetical** scenarios chosen in advance
Demo investors	**Invented** (the team's personas); no real person's data is in the repository
C1-C5 limits	**Placeholders** pending a bank's own table (a banner says so)
Blockchain seals	**Real transactions on the Sepolia test network** for invented investors
Orders	**None**: no order is ever placed; the broker link is read-only

10. Evidence snapshot

About **3,540 automatic tests** (3,538 passing in the last complete run on 20 Oct; the one failure is a timing test of the news page, unrelated to any investment rule); hostile-input suites; property sweeps (300 investors through the category rule, 5,000 random trades); mutation checks; live red-teams; a read-only deploy check; real on-chain seals; a fourteen-investor reading of every investor-facing sentence. Details and re-run commands: `docs/judges/04_TESTING_AND_EVIDENCE.md`.

11. Limits, honestly

Category limits are provisional; goal chances depend on past data; Yahoo-sourced snapshot prices are for the competition only; e-mail confirmation uses a small shared sender; the blockchain is a test network; Chinese-language advice and more brokers are roadmap. The full list: `docs/judges/06_LIMITATIONS_AND_ROADMAP.md` and `docs/SECURITY.md` section 4.

12. Team and way of working

The people and the Claude sessions: Savas (manager), Bideci (Supabase and integration), Aslan (news briefing and the on-chain seal), Zerya (the goal simulation and regime model) and the Claude sessions Onur (integration, quant), Umut (frontend), Damla (data and tests), Bartu (questionnaire, behaviour, evaluation). Rules that kept it safe: own git worktrees, a full automatic test suite before every merge, secret

scans on every diff, one integration branch, an independent check at every boundary, and a written team tracker.

Glossary

Category (C1-C5): the person's risk grade, C1 most careful. **R-level (R1-R5)**: a product's risk grade. **Attitude to risk**: how a person says and plays they feel about risk. **Capacity**: how much loss their finances can absorb. **Risk Engine**: the independent final check. **Drawdown**: the biggest fall from a peak. **Effective holdings**: how many equal-sized funds a portfolio is as spread out as. **Natural frequency**: "N in 100" instead of a percentage chance. **RLS**: Row-Level Security, the database rule that lets each user see only their own rows. **Seal**: a fingerprint of the advice record published on a public test chain.